– Describe your overall approach to implementing the game,

Our approach to implementing this game was to have:

Main class, GameHandler class, UIHandle, and Round class. These classes work together to run the main menu, as well as each round of the game.
We also have a Map class, which has Tile class objects, the tiles are composites of the map class.

We used an abstract factory design to implement the entities in the map. The “factory” part is the gameObjectHandler class, which is extended by ChestHandler, ExitHandler, GemHandler, MonsterHandler, and TrapHandler. These handlers create and update their corresponding GameObjects. The GameObject class is the “product” part of the design, it is extended by Chest, Exit, GameObject, Gem, Monster, Player, and Trap.

For implementing the monster behaviour, we had 2 states in which the monster can be in, roaming and chasing. When the monster is in roaming state, it will pick a random direction in regards to where it is currently facing and will constantly search for the player using a form of ray casting (not a true raycast). If the monster spots a player, it will enter in the chasing state and construct a path using breadth-first-search (BFS) to where the player was last seen and move to that position. When it has reached where the player was last seen and can not see the player, it will return to its roaming state.

The UIHandle class handles everything we see and interact with. It renders which draws the map, player, monsters, etc. It uses JavaFX built in canvas and graphics context to draw all those things. The class also listens for keyboard events for the player movement, which the GameHandler reads to move the player. Lastly, the main class has certain styling and animations to make the game look a bit better.

– State and justify the adjustments and modifications to the initial design of the project (shown in class diagrams and use cases from Phase 1),

Our design is very different from our initial design, the differences are as follows:

- We added the map and tile classes, this was due to an oversight during the initial planning process.
- We implemented the factory method for creating gameObjects. During the planning process, we underestimated the scale of creating all these entities, and did not have handlers.
- We implemented a collision handling and pathfinding class and reworked the monster class to use these classes

– Explain the management process of this phase and the division of roles and responsibilities,

We managed this phase of the project by initially meeting on a video call to discuss the division of responsibility, and for further discussion between each other we just messaged our group chat.

Initially we divided it up as follows:

Ben: the GameObject class and its children.
Alex: the round class
Smit: User interface and main class
Daniel: Map and tile classes

However, the scale was much larger than just the classes initially discussed, so we worked together on the rest of the classes.

– List external libraries you used, for instance for the GUI and briefly justify the reason(s) for choosing the libraries,

We used JavaFX 23 for the GUI, rendering, scene management, and input handling. We used this because it is natively supported by Maven and it's good for 2D graphics. Also JavaFx has all the built in things for animations.
We also used Maven for the build and dependency management to simplify the compilation and version control of the other external library.

– Describe the measures you took to enhance the quality of your code,

Some measures we took to enhance the quality of our code are as follows:

- Clean javadoc comments
- We have minimal coupling
- Ensure our classes have high cohesion
- Adopted design patterns to where it is needed

– Discuss the biggest challenges you faced during this phase.

The biggest challenge of this project was communicating design ideas and changes with group members while we all have very busy schedules, allowing for little time to discuss.

Another challenge was in the code design itself, the game was ending abruptly every time we started it. This was due to an index out of bounds exception in the grid instantiation, which was initializing the grid in regards to width x width instead width x height.

Another challenge was implementing the monster's behaviour and movement, we underestimated how complex the implementation was going to be to add features like monster vision and pathfinding.